

Devolución TP1

Grupo 1

Demo

- pageSize no se respeta en el explore. Solo aceptan las que están en el dropdown (no tiran ningún error).
- Manejo correcto de números de página random
- UX:
 - La foto del restaurante en el detail se pixela mucho. Opción: se pueden subir dos fotos (una horizontal y una vertical)
 - Date picker con logo negro no se ve bien
 - Si yo me deslogueo como user normal, y logueo como restaurant, me redireccionan al feed
 - My restaurants y my profile no están seleccionados en el navegador
 - Discover More Lists no es nada claro con el focus
- Bien que el verify te deja iniciado sesión.
- Ante consultas de la cátedra acerca del funcionamiento de la página, hubo varias instancias en las que no supieron responder. **No puede suceder nunca que no sepan como funciona la aplicación, el uso de un agente en forma responsable requiere una persona que entienda, valide y se haga responsable por los cambios introducidos.**
 - No supieron responder cómo la página determina si un restaurante está "abierto en el momento" o no.
 - En general no tienen claro lo que hacen con javascript en el front end.
 - La búsqueda en las listas trae cosas que no debería traer. No saben sobre qué campos de información están filtrando.
- No respetan el locale del usuario para los mails
- Manejan bien permisos. No puedo eliminar un restaurante con curl y esas cosas.
- Escapan bien los caracteres para SQLI
- En el home tienen un botón de Create Account aunque ya esté logueado que me lleva a la página de register.

Código

- En el `ReservationServiceImpl` el método `completeExpiredConfirmedReservations()` obtiene los IDs expirados con una consulta y luego llama a `reservationDao.findById(reservationId)` dentro del loop por cada ID. Patrón N+1: el DAO debería retornar directamente los

objetos `Reservation` completos desde `findExpiredConfirmedIds`, eliminando las N consultas adicionales. **Esto es un error conceptual grave.**

- En el `ReviewServiceImpl` se ejecuta `userDao.findById(userId)` para validar el rol reviewer en cada creación de review. Consulta redundante: el endpoint `POST /restaurants/*` ya está restringido a `isPureReviewer` en `WebAuthConfig` por lo que Spring Security garantiza que solo reviewers llegan al método.
- En el `ReservationServiceImpl`, el método `syncOwnerSpecialList()` carga todos los restaurantes del owner y filtra `isApproved()` e `isOpen()` en Java, además de ordenar por nombre e ID también en Java. El filtrado y el ordenamiento deben resolverse en SQL mediante cláusulas `WHERE` y `ORDER BY` en el DAO, no en memoria. **Esto es un error conceptual grave.**
- En `RestaurantListController`, `currentUser.hasRole("ROLE_ADMIN")` se evalúa directamente en el cuerpo del controller y se pasa como booleano al servicio. Aunque el resultado se delega, la inspección del rol ocurre en el controller. El chequeo debería delegarse al service del mismo modo que realizan el check `isReviewer()`.

Seguimiento

No resta puntos por seguimiento.

Nota

5

Grupo 2

Demo

- Cuando verificas la cuenta el mensaje indica que hagas login pero realmente ya realizan el auto login
- En mi perfil, en la tab de ventas, faltaría un CTA para publicar
- Cuando actualizo mis datos en mi perfil si hay un error no vuelve a la tab seleccionada
- En el reset password me piden los datos de mi email cuando ya estoy logueado
- Estaría bueno sumar el comprobante de pago y no solo un botón
- ¡18n no cambia de idioma
- Si hago una búsqueda, quedo un placeholder de interpolación `{0}`
- Cuando solicite el vinilo no le llego al seller un mail
- Los mails tienen un link con "token", esto debería sacarse del login del user
- Numero de pagina negativo da 500, debería ser 400
- El ban no tiene confirmación
- El ban solo valida que no te puedas loguear, si el user se mantiene logueado puede seguir publicando

Código

- Su pom de webapp declara la dependencia a services sin definir scope `runtime`.
- Todos los módulos hijos redeclaran `<version>1.0-SNAPSHOT</version>` y `<groupId>ar.edu.itba.paw</groupId>` en lugar de heredarlos del padre.
- Agregan `AGENTS.md` al `.gitignore`. **Esto es un error grave, ya que evita que todo el equipo siga el mismo workflow, contexto y reglas.**
- Tienen manejos de autenticación y permisos en los controllers. Por ejemplo, `UserController.login` valida que el usuario no esté autenticado, o `ProductController.editProductForm`. Estas lógicas deberían estar definidas a través de spring security.
- Tienen duplicado el código de `refreshAuthenticationPrincipal`.
- Tienen lógica de negocio en los controladores. Por ejemplo, es el `ProductController` quien hace llamados para la creación del producto y la posterior creación de las imágenes, debería estar encapsulado en la creación del producto. **Esto es un error conceptual grave.**
- Tienen validaciones en los controladores que deberían ser custom validators en los formularios. **Esto es un error conceptual grave.**
- Configuran `ThreadPoolTaskExecutor` pero sin `setWaitForTasksToCompleteOnShutdown` o `setRejectedExecutionHandler`.
- Tienen varias queries N+1. Por ejemplo, en `HomeController`, se traen uno por uno los `summaryForSeller` cuando podrían hacer un `WHERE id IN ...`. **Esto es un error conceptual grave.**
- Tienen anotaciones `@Transactional` que podrían ser `@Transactional(readOnly=true)`.
- Su `ProductSearchCriteria` usa la constante 12 pero aparece como magic number, mejor si la extraen a una constante.
- Para banear a un usuario, se traen sus primeros 1000000 productos (cantidad hardcodeda) e iteran por cada una de ellas para esconder el producto y borrar los reportes asociados. Deberían tener métodos en los otros servicios que les permitan actuar en base al usuario sin necesitar iterar. **Esto es un error conceptual grave.**
- Los mails obtienen el locale a través de `LocaleContextHolder.getLocale()` (en un contexto Async) sin respetar el locale del destinatario. **Esto es un error conceptual grave.**
- `ProductDao` es accedido directamente tanto por `ProductServiceImpl` como por `ReportServiceImpl`.
- Su `RowMapper` de Product hace una subquery para obtener las categorías por cada producto. Deberían hacer un JOIN y agrupar en memoria.
- Anti intuitivamente, no es `CategoryJdbcDao` la clase encargada de manipular la tabla `products_categories`, sino que se encarga sólo de las lectura y es `ProductJdbcDao` quien se encarga de las inserciones. **Esto es un error conceptual grave.**

- Tienen texto hardcodeado en sus jsps. El title de `product-detail.jsp` como `"Vinyland | ${product.title}"` y la hint de "Sin imagen".
- En sus modelos tienen campos como boxed primitives a pesar de que sus tablas los definen como `NOT NULL`.
- Los campos de `Token` son prácticamente final pero no están declarados como tales.
- `PurchaseStatus` no tiene sus mensajes internacionalizados.
- En `PurchaseJdbcDao` combinan el status, el token del comprador y el token del vendedor en un mismo campo string en vez de guardarlos por separado.
- Su definición del logger para `ar.edu.itba.paw.services` repite lo ya definido para `ar.edu.itba.paw`.
- Para la autenticación de las compras se manejan con tokens, ignorando el usuario autenticado actual. Cualquier usuario con el link puede modificar el estado.
- Tienen parámetros de los controladores como strings que ya podrían tener el tipo necesario y así ahorrarse parseo.
- No tienen tests para `ProductServiceImpl` a pesar de ser una clase con mucha lógica. **Esto era pedido por enunciado.**
- Sus recursos de mailing se encuentran distribuidas entre webapp para los templates y services para las localizaciones.
- Tienen muchos tests que no son unitarios: por ejemplo `ReviewJdbcDaoTest` crea entidades y luego usa un find del mismo dao para retornarlo. En algunos casos incluso dependen de otros daos. **Esto es un error conceptual grave.**

Seguimiento

No resta puntos por seguimiento.

Nota

3

Grupo 3

Demo

- La hint de la mínima oferta actual para una subasta se puede simplificar
- Sería útil autocompletar el campo de puja con el monto mínimo
- Bien que no revelan si la cuenta existe en el cambiar contraseña
- Bien que el verificar cuenta deja logueado
- Estaría bueno agregar una preview de frontend al actualizar la foto de perfil
- Al ver mis subastas, clicar en la card me debería llevar al detalle de la misma

- "1 reseñas" cuidado con la localización
- Falta validar que los rangos de fechas sean válidos (fecha "desde" previa a "hasta")
- La validación del precio inicial de la puja no permite que el precio sea mayor o igual al precio original, pero el mensaje de error sólo dice que no puede ser mayor
- Al crear un producto, valida ambos campos completados (para producto normal o subasta)
- La puja puede llegar a exceder el precio original
- Al pujar en una subasta, volver atrás me lleva a la misma página.
- Estaría bueno agregar un botón de limpiar la búsqueda cuando no hay resultados. Y que limpie también el campo de texto
- Métricas interesantes para el negocio: monto ahorrado/no perdido gracias, productos con mayores pujas
- Al finalizar una subasta, el producto ya no se puede ver desde el lado de los clientes. Mejor que aparezca como "subasta finalizada" como tienen para los negocios.
- "This code belongs to a reservation from another commerce." No hace falta aclarar que es válida pero de otro comercio.

Código

- `logback-test.xml` vive en `src/test/resources` y no en `src/main/resources`. Esto hace que si el entorno de desarrollo no incluye el classpath de test, entonces se utilice la configuración de producción.
- La dependencia `mockito-junit-jupiter` declarada en el `<dependencyManagement>` del POM padre no tiene `<scope>test</scope>`, así que cualquier hijo que la incluya la arrastra al runtime de producción.
- `WebAuthConfig` no está declarado en `contextConfigLocation` de `web.xml`. Funciona porque queda dentro del `@ComponentScan("ar.edu.itba.paw.webapp.config")` de `WebConfig`, pero la dependencia no es declarativa y un lector tiene que ir a `WebConfig.java` para enterarse.
- Tienen un default hardcodeado para la `remember-me` key (`"resqfood-remember-me-secret"`), y la clave es débil. Si la propiedad real no se carga (env corrupto, env.properties faltante en deploy), el fallback es público en el repo y se pueden forjar tokens. **Esto es un error grave.**
- El `passwordEncoder` no se asocia explícitamente al `AuthenticationManagerBuilder`. Funciona por autowire vía `ObjectPostProcessor`, pero un reviewer no ve la conexión leyendo `WebAuthConfig`.
- Validan ownership de los recursos en la capa de servicios (`PackServiceImpl.resolvePackForDirectEdit`, `AuctionServiceImpl.cancelAuction`, etc.) en lugar de usar la capa de seguridad. No usan `@PreAuthorize` / `PermissionEvaluator`. Cada método nuevo depende de que el dev se acuerde del check. **Esto es un error conceptual grave.**
- En `EmailVerificationController`, después de verificar el email hacen login programático escribiendo el `SecurityContext` en la sesión actual sin invalidar la sesión previa.

- `sessionManagement().invalidSessionUrl(...)` no está configurado: cuando expira la cookie, Spring redirige al default en vez de a una URL de error explícita.
- Hay lógica de negocio en controllers. Hay varios controllers, un ejemplo claro es el `prefetchClients` que utilizan repetidas veces, esto debido a que su modelo almacena el ID de un client y no un Client en si, a pesar de que lo usan ampliamente en su pagina. Esto causa que tengan que estar buscando el client repetidas veces y lo terminan haciendo desde el controller, cuando lo mas fácil sería que ya venga en el modelo que lo contiene. **Esto es un error conceptual grave.**
- Usan `if (access instanceof CommercePackAccess.NotFound)` y `instanceof CommercePackAccess.ForbiddenAuction` para decidir el HTTP status dentro del controller. Eso es responsabilidad del `GlobalExceptionHandler`: el service debería lanzar excepciones de dominio y el handler las mapea centralizadamente.
- En `ImageController`, si la imagen no existe devuelven `200 OK` con un placeholder SVG. Eso rompe la semántica HTTP: browsers cachean el placeholder como si fuera el recurso real, y la decisión de fallback debería ser del cliente. Lo correcto es retornar un `404`.
- El base URL para los mails (`appBaseUrl`) se calcula desde el `HttpServletRequest` en cada controller y se pasa al service por parámetro. Debería ser una property de la app (`@Value("${app.base-url}")`) — esto evita un Host header injection en los links del mail.
- Hay varios N+1 a lo largo del código: `closeExpiredAuctions` ; `CommerceDashboardController.prefetchClients` ; `CommerceDashboardController.products` ; `ReservationListModelBuilder.buildCommerceView` ; y `ReservationListModelBuilder.buildClientView` . En todos estos casos el patrón correcto es realizar una sola query que traiga los datos necesarios utilizando JOIN. **Esto es un error conceptual grave.**
- El `RowMapper` de `PackJdbcDao` dispara una sub-query a `pack_tags` por cada fila del result set. En un RowMapper no es correcto hacer subqueries; lo correcto es un `JOIN` con `pack_tags` y agrupar tags en memoria.
- Tiran `IllegalArgumentException` e `IllegalStateException` del JDK como si fueran excepciones de negocio. Mezcla precondiciones rotas (errores de programación) con reglas de negocio violadas (errores del usuario). El proyecto sí tiene algunas excepciones custom (`CommerceReviewException`, `BidPlacementException`), pero los services centrales (`ReservationServiceImpl`, `AuctionServiceImpl`, `UserServiceImpl`) usan las del JDK.
- Tienen casi todos los métodos `@Transactional` pero los de solo lectura no usan `readOnly=true`.
- La interfaz `EmailVerificationTokenService` debería llamarse `VerificationTokenService`: que el canal sea email es un detalle de implementación. Mañana podría sumarse SMS o push y la interfaz quedaría mal nombrada.
- Los `RowMapper` de varios DAOs están definidos como lambdas asignadas a campos de instancia en el constructor, no como `private static final RowMapper<X>`.
- `PackJdbcDao` y `AuctionJdbcDao` usan funciones exclusivas de Postgres (`ILIKE`, `SPLIT_PART`) en las queries de filtro, pero los tests usan HSQLDB y siempre pasan `query=null` / `timeRanges=null`. El 100% del filtrado queda sin cobertura.

- El modelo `User` incluye `password` en el `toString`. Si en cualquier flujo se logea un `User`, el hash BCrypt termina en los logs y queda disponible para ataque offline.
- Sería correcto agregar `equals/hashCode` en los modelos asociados al ID únicamente.
- En `register.jsp` usan `http://java.sun.com/jstl/core_rt` en lugar de `http://java.sun.com/jsp/jstl/core` (URI estándar JSTL 1.2 EL).
- En `products.jsp` hay texto en español hardcodeado (`Mis productos`) sin pasar por `<spring:message>`.
- `profileView.jsp` cierra `</body></html>` sin invocar `<paw:footer/>`. Todos los demás JSPs sí lo invocan.
- Los filtros del catálogo de packs se reconstruyen manualmente con un `StringBuilder` (25+ líneas serializando `q`, `tags`, `types`, `sort`, `location`, `timeRange`). Si envolvieran el `searchBar` y los filtros en `<form action="/packs" method="get">`, el browser serializaría todo automáticamente y desaparece todo el `baseUrlBuilder`.

Seguimiento

No resta puntos por seguimiento.

Nota

4

Grupo 4

Demo

- La pantalla de "Correo verificado" tiene un botón que sobrepasa los límites de su contenedor.
- El código de colores por estado del protocolo no es intuitivo (ie: completado es naranja)
- En el detalle de un protocolo las etiquetas no son clickeables
- La terminología de "seguir" protocolo es poco clara cuando la página tiene tantos componentes sociales... ¿seguir es ver novedades y resultados o es "unirme" cuando reclutan?
- Cuando empiezo a "seguir" a un protocolo empieza a hablarme de "suscripción". La terminología es inconsistente.
- Demasiados botones, demasiada info junta... es muy poco clara la página y difícil de seguir.
- Es posible "seguir" multiples protocolos en paralelo, incluso cuando hacen a lo mismo... ¡los resultados no son independientes!
- ¿Por qué el progreso no arranca hasta que arranca la cohorte? Entiendo que no haya data hasta tener un número de muestras, pero no es intuitivo el que empiezan todos juntos.

- El explorador de sugerencias muestra una lista visualmente cortada
- Las categorías de protocolos y métricas con independientes y no se hablan una con la otra.
- La creación de protocolo muestra filas incompletas de protocolos (con huecos en blanco), pero igual el resultado se pagina. Ocurre lo mismo en múltiples pasos del wizard.
- La aplicación tiene múltiples problemas de encoding, rechazando valores como tildes, y mal formando las respuestas.
- La carga rápida sigue mostrándose al usuarios incluso con 0 pendientes.
- Todo el flujo es confuso y sobrecargado de información. No está orientado a resolver el problema base que ellos mismos declaran (saber si algo que veo en redes funciona o no); por el contrario, se enfoca en descubrir y registrarse en protocolos que aún no comenzaron.
- Recibo cada día correos recordando que cargue los datos de un mismo protocolo por triplicado en el mismo minuto.

Código

- El módulo `webapp` declara dependencias que no están definidas en el pom padre, y hasta configura versiones para módulos de spring en lugar de heredarlo del pom padre.
- El repositorio muestra claras señales de abuso de agentes sin un control apropiado. Por ejemplo, múltiples clases con más de 1000 líneas de código (y en algunos casos superando las 4500); paquetes que mezclan decenas de clases con objetivos diversos; funciones individuales que superan las 150 líneas de código, etc. **Estos son errores graves.**
- Los controllers realizan en forma manual controles de acceso. Esto es responsabilidad de Spring Security.
- Los controllers tienen lógica de negocio, en vez de delegar sobre un Façade en la capa de servicios. **Esto es un error conceptual grave.**
- La aplicación hace uso extensivo de magic numbers.
- Los controllers hacen validaciones a mano por no hacer un uso efectivo de formularios y Bean Validation.
- Los tests usan `doAnswer` para capturar las invocaciones y valores pasados al mock. Esto es equivalente a usar `Mockito.verify` para validar la implementación, y no el comportamiento de la clase siendo testada. **Esto es un error grave.**
- Los tests no son unitarios. Múltiples tests hacen uso de distintos métodos de la class under test para el setup o la validación de escenarios. **Esto es un error conceptual grave.**
- La gestión de contexto para el agente es incorrecta.
 - Existen múltiples archivos solapados, como `CLAUDE.md` y `AGENTS.md`. Debiera haber uno sólo (idealmente el segundo, que es agnóstico de la herramienta)

- Estos archivos son extremadamente extensos, y con demasiada información irrelevante o inespecífica.
- Tanto estos archivos como web-hooks se explayan en como mantener estos archivos idénticos. Esto no sólo es inútil ya que no debieran existir los dos, sino que evidentemente no funciona, ya que los 2 archivos en el commit entregado son distintos.
- Documentar cosas como "bibliografía", el "flujo viejo de pseudo-login" o la duración de los sprints son irrelevantes para el agente, ya que no es información accionable. Duplicar bloques de XML del pom no suma valor, solo introduce contradicciones cuando uno cambia.
- Lo mismo aplica a la estructura detalla de archivos, en lugar de concentrarse en definir las responsabilidades de cada módulo o las reglas a seguir en la implementación.
- Lo mismo aplica respecto del schema de la base de datos, que puede ser obtenido directamente de observar un schema SQL.
- Hay contradicciones internas en el contexto provisto, por ejemplo, al indicarle primero que sólo existen 2 roles, "USER " y "ADMIN" y luego decirle que no debe haber un mapeo 1:1 con tipos de usuarios:

Roles: `ROLE_USER` (todos), `ROLE_ADMIN` (gestión de catálogos, moderación).
 Modelar control de acceso con RBAC real: preferir roles sobre "tipos de usuario" nominales, y definir roles por conjuntos de operaciones afines.
 Un mismo usuario puede tener múltiples roles simultáneamente; evitar asumir una relación 1 usuario = 1 tipo.

- La capa web accede en forma directa a la base de datos violando el modelo de capas jerárquicas establecido. **Esto es un error conceptual grave.**
- Consistente con el AI slop que abunda en el proyecto, hay muchísimo código innecesario. Se les recuerda fuertemente a los alumnos la necesidad de revisar y validar en forma exhaustiva todo el código producido por el agente y procurar entender lo que hace, el por qué, y la necesidad que satisface el mismo. Por citar algunos ejemplos (lista no exhaustiva):
 - `RequestParamParsers` es innecesario, Spring WebMVC es capaz de proveer argumentos en el tipo de dato deseado; y bean validation puede validar los valores (que si no se cumplen son un error 400)
 - `DatabaseEncodingValidator` es innecesario, ya que la base de datos provista es conocida y su configuración no va a cambiar.
 - La reimplementación de remember-me via `DeviceScopedPersistentRememberMeServices` y `DeviceScopedJdbcTokenRepository` es completamente innecesaria, ya que la implementación default es correcta y tiene el remember-me scoped a un profile del browser.
 - `RequestCorrelationFilter` es completamente innecesario. Existiendo una única instancia del server, el thread name es más que suficiente para correlacionar entradas de logs; como discutimos en clase.

- `CurrentUserSynchronizationFilter` es innecesario. Termina usando magic strings para definir keys dentro de la sesión para que los controllers y JSPs puedan acceder a esta información cuando sería más apropiado usar o un `@ModelAttribute` en un `@ControllerAdvice` (visto en clase). Aún más, esta solución no sólo es menos segura y más compleja, además requiere que `User` sea `Serializable`.
- `AuthController.resolveRegisterError` depende de magic strings en el mensaje de excepciones lanzadas en otras capas de la aplicación. Esto es frágil y difícil de mantener.
- La aplicación loguea información en forma desmedida, guardando 30 archivos en cada appender, y logueando absolutamente todo a nivel `INFO` a `STDOUT`. Esto se pidió explícitamente evitarse explicándose en clase los motivos por lo que nunca debe hacerse esto. **Esto es un error grave.**
- Como parte de la misma tarea, crearon un `PasswordResetTokenDao` y `EmailVerificationTokenDao` que son idénticos. Estos incluyen un DTO embebido en la interfaz (y no como parte del dominio), que también es idéntico. Ambos sirven el mismo propósito, y es innecesario mantenerlos separados.
 - Toda la lógica de `SessionInvalidator` creada como parte de este mismo stream es innecesaria.
- Sincronizan en forma manual, dependiendo de `WebUtils.getSessionMutex`. En una aplicación web Java jamás debiéramos gestionar la sincronización en forma manual. **Esto es un error conceptual grave.**
- Los servicios usan en forma indiscriminada otros servicios o directamente los DAOs incluso de otras entidades sin un criterio. Lo correcto es sólo usar en forma directa el DAO de la propia entidad, y depender de los servicios para las otras entidades que se precisan.
- El envío de recordatorios de carga de datos no manda un mail por usuario, sino un mail por cada suscripción del usuario a un protocolo pendiente.

Seguimiento

No resta puntos por seguimiento.

Nota

3

Grupo 5

Demo

- Mucho texto, la gente no lee tanto.
- Sign In ⇒ Me logea, pero no dispara la acción que había querido hacer.
- El locale se respeta en los labels de los forms, pero no a la hora de ingresar datos numéricos (EX: muestra 50,000 pero al escribir me sale 50000)

- Personalmente, lo pondría como paso MANDATARIO cargar al menos una dirección. Al menos como cliente, prácticamente no te deja operar la página.
- Intervalos fijos, Siempre de a 30 min. Quizá dependiendo el servicio se puede quedar corto (Ej: Instalar algún aire acondicionado). Por los slots de reserva. Está hardcoded en 30 min de bloques para todos los servicios.
- Bien de features.
- El flujo principal está bien pensado y se entiende.

Código

- Definen tanto dependencias de JUnit 5 como de JUnit 4, en lugar de definir y usar una sola. **Esto es un error conceptual grave.**
- Autorización delegada al método vía anotaciones con un filter chain abierto deja endpoints sin anotación completamente públicos (Ej: `webapp/.../WebAuthConfig.java:46-65` con `anyRequest().permitAll()`; endpoints sin `@PreAuthorize` como `GET /contractors/{id}` en `ContractorController.java:164` y `POST /register/contractor-profile` en `ContractorController.java:123`). Lo correcto es centralizar toda la configuración en `WebAuthConfig`
- Múltiples JSPs renderizan sin `<c:out> / escapeXml` (en texto, atributos `value` y `style`) habilitando XSS reflejado (Ej: `webapp/.../contractor-profile.jsp:391` con `${question.client.firstName} ${question.client.lastName}`). **Esto es un error conceptual grave.**
- Orquestación de múltiples services desde el controller en lugar de encapsularla bajo un verdadero Façade — cada service aporta su `@Transactional` por método, pero al no haber tx única el conjunto queda sin atomicidad ante fallo parcial (Ej: `webapp/.../ContractorController.java:130-144` `contractorProfileSetupPost`). **Esto es un error conceptual grave.**
- Los controllers implementan explícitamente reglas de control de acceso (Ej: `webapp/.../UserController.java:161-162` con `hasContractorRole(authentication)`)
- Extracción del user autenticado por cast manual repetida en múltiples controllers en vez de centralizarse en un `@ControllerAdvice` o usar el `@AuthenticationPrincipal` idiomático de Spring Security (Ej: `webapp/.../ContractorController.java:178` con `(AuthUserDetails) authentication.getPrincipal()`)
- Binders de forms acoplados al request HTTP para inferir ruta mezclan presentación con lógica de binding (Ej: `@ModelAttribute` en `webapp/.../RequestController.java:78-88` `loadRequestForm` recibe `HttpServletRequest`)
- Validaciones manuales (`trim`, parseo de enum a mano, normalización `toUpperCase/toLowerCase`) en controllers en vez de Bean Validation en los forms (Ej: `webapp/.../UserController.java:183` con `userForm.getRole().trim().toUpperCase(Locale.R00T)` — patrón repetido en `ImageController.java:80-81`, `UserController.java:131`, `ContractorController.java:220`, `RequestController.java:586`)
- Múltiples métodos de escritura en services no están anotados con `@Transactional`, pudiendo dejar la BD inconsistente ante fallos parciales (Ej:

- `services/.../UserServiceImpl.java:45-52` `create` invoca `userDao.create` + `createToken`)
- Métodos solo-lectura sin `@Transactional(readOnly=true)` desaprovechan optimizaciones del TX manager y no documentan intención (Ej: `services/.../UserServiceImpl.java:30-42` con `findById/findByEmail/existsByEmail` — patrón repetido en `ReviewServiceImpl.java:59-83` y `ContractorServiceImpl.java:69-82`)
 - Services que dependen directamente de múltiples `Dao` en vez de delegar a otros services rompen la composición de la capa de servicios (Ej: `services/.../RequestServiceImpl.java:46-54` inyecta `RequestDao` + `AddressDao` + `QuoteDao` ; `ReviewServiceImpl.java:42-47` inyecta `ReviewDao` + `RequestDao` ; `AddressServiceImpl.java:31-34` inyecta `AddressDao` + `RequestDao`)
 - Múltiples instancias de patrones N+1 en services degradan performance al iterar invocando DAO por elemento o materializar `findAll()` por enriquecimiento (Ej: `services/.../ReviewServiceImpl.java:178-182` con `userService.findById` por cada review)
 - Lookup/filtrado en memoria sobre `findAll()` de catálogos completos no escala si los datasets crecen (Ej: `services/.../AddressServiceImpl.java:145-150` materializa `neighborhoodService.findAll()` para armar un `Map<Long, Neighborhood>` ; mismo patrón en `ContractorServiceImpl.java:355` con `tradeService.findAll().stream()` — conviene `WHERE id IN (...)`).
 - DAOs que operan sobre tablas ajenas a su entidad rompen la exclusividad de tabla por DAO y generan múltiples puntos de acceso a la misma tabla (Ej: `persistence/.../ContractorJdbcDao.java:150-156` ejecuta `DELETE FROM contractor_trades` y `DELETE FROM contractor_neighborhoods` , tablas de relación que conviene encapsular en un DAO específico). **Esto es un error conceptual grave.**
 - Mensaje crudo de excepción interna enviado al usuario en vista de error puede filtrar stack traces o detalles del framework (Ej: `webapp/.../GlobalExceptionHandler.java:111` con `mav.addObject("errorDetails", detailMessage(ex.getMessage()))`)
 - Modelos sin `equals/hashCode` por ID hacen que dos instancias con mismo `id` no sean `equals` , rompiendo `Set / Map / contains` en lógica de negocio (Ej: `models/.../User.java` , `Address.java` , `Contractor.java` , `Request.java` , `Review.java` no implementan ninguno)
 - Modelos con Lombok sin excluir explícitamente los campos sensibles del `toString` arriesgan filtrar credenciales si algún flujo logea el objeto (Ej: `models/.../User.java:1-18` con `@RequiredArgsConstructor` sin `@ToString(exclude = "passwordHash")`)
 - Los tests NO son unitarios, hacen múltiples cambios por test y chequean el estado utilizando las funciones del negocio. **Esto es un error conceptual grave.**
 - Tests con `AtomicBoolean / AtomicReference` para simular `Mockito.verify()` / `ArgumentCaptor` testean implementación.
 - Cobertura de tests muy pobre. **Esto era pedido por enunciado.**

Seguimiento

No resta puntos por seguimiento.

Nota

3

Grupo 6

Demo

- Los géneros no están en orden alfabético
- Cuando me intercepta y me pide registrarme y hacerlo, me vuelve a mandar a la home en lugar de a la acción que quise realizar
- No hay forma fácil de marcar un libro como leído.
- Si no tengo clubes, la sección vacía "Mis clubes" sólo oculta la sección de "Descubrir Clubes" que es la que quieren promover
- Ante consultas de la cátedra acerca del funcionamiento de la página, hubo varias instancias en las que no supieron responder. **No puede suceder nunca que no sepan como funciona la aplicación, el uso de un agente en forma responsable requiere una persona que entienda, valide y se haga responsable por los cambios introducidos.**
 - No supieron responder sobre el rate limiter introducido (en que capa está, como deshabilitarlo, etc)

Código

- Los controllers tienen mucha lógica de negocio. También hay chequeos de acceso orquestados a nivel controller en lugar de delegados a Spring Security, mezclando responsabilidades. **Estos son errores conceptuales graves.**
- Tienen vulnerabilidades XSS en la search bar de comunidades, por no escapar contenido. **Esto es un error conceptual grave.**
- Los tests de persistencia no son unitarios en varios casos: preparan el estado usando métodos del mismo DAO que después testean. **Esto es un error conceptual grave.**
- El uso de Transactional en general está OK hay algunos casos en los que falta @Transactional(readonly=true).
- Usan configuración de DEBUG en producción para los logs a nivel aplicación.
- Buena cobertura de tests en la capa de servicio y son unitarios. Están bien salvo:
 - `testIsValid` no es unitario porque hace 3 pruebas distintas (Sugerencia: usar tests parametrizados)
 - los tests de email no testean lo que dicen testear, no hay asserts, considerar lanzar excepciones o devolver algún resultado para testear.
- Hay consultas N+1 en varias instancias. `ClubPollController.resolvePreselectedBooks` itera los bookIds del form y llama `bookService.findById(...)` para cada uno siendo sumamente ineficiente. Este patrón se repite en varios métodos.

- Aplican Rate limiting frente a logins por ip, incluso exitosos. Lo cual puede bloquear logins de personas metiéndose bajo la misma ip (Ej: un aula, universidad, etc...). Para peor, las estructuras usadas guardan en memoria información a la eternidad hasta recibir un nuevo request de la misma IP. **Estos son errores conceptuales graves.**
- Los endpoints de posts se permiten públicamente, pero el controller dereferencia `currentUser.getUser()` sin validar autenticación. Un anónimo puede llegar al método y producir un error en vez de una denegación declarativa.
- La key de remember-me esta hardcodeada como `"paw-web-secret"`. Debe venir de configuración externa y ser robusta; si queda en el repo, los tokens son forjables.
- El controller construye URLs absolutas para mails desde el `HttpServletRequest`. Esto debería ser una property de la app, no depender del host recibido en el request que es susceptible a forging.
- Hay validación y normalización manual de inputs en controllers (`trim`, limites de paginación, validaciones de schedule), en lugar de concentrarlo en forms/Bean Validation. **Esto es un error conceptual grave.**
- Hay endpoints que devuelven strings de un protocolo inventado (`OK:/landing`, `ERROR:...`) con `@ResponseBody` en vez de manejar el flujo con redirección y usar el protocolo HTTP. **Esto es un error conceptual grave.**
- Las búsquedas con `LIKE` agregan `%` al input del usuario pero no escapan comodines como `%` y `_`. El usuario puede alterar el patron de búsqueda.
- Algunos `RowMapper` están definidos como lambdas en campos de instancia o en el constructor, no como `private static final RowMapper<X>`. Eso mezcla estado del DAO con mapeo reutilizable.
- Hay texto visible hardcodeado en JSP/tag files, por ejemplo labels `Token`, `Avatar` y `Placeholder`. Todos los textos visibles deberían salir de `messages.properties`.

Seguimiento

No resta puntos por seguimiento.

Nota

3

Grupo 7

Demo

- Call to action en details requiere scroll, tendría que estar visible apenas ingresas
- Además la búsqueda de texto libre no tiene mucho sentido buscando cocheras
- El register no hace login.

- Algunos polígonos están mal proyectados
- La UX al reservar por primera vez un espacio no es buena, ya que te ofrece un form que siempre falla.
- Hay modales con mucha UX. Para forms largos con varios pasos o tabs no es correcto usar modales.
- Es posible causar un 409 creando un espacio
- Likear un posteo redirecciona mal (perdés el contexto del url)
- La UX expone como like la funcionalidad de marcar como favorito, lo que confunde al usuario
- Falta flujo de cancelar a travez de la página (del lado del owner del espacio)
- El flujo de generar un espacio me obliga a salir de la página (para setear lat y long). Debería ser todo vía la página.
- Puedo poner cualquier dimensión (de las predeterminadas) y luego cualquier ancho profundidad. Sería preferible que no deje poner un tamaño en el cual no quepa el auto seleccionado en cuestión
- Botón de reservar esta desactivado, pero para la UX no parece
- No se puede editar fechas de disponibilidad
- Paginación incompleta, debería tener botón de next y prev, tamaño de página e idealmente ir a la ultima / primera
- No se puede cambiar el locale manualmente desde la pagina
- Reservations debería ser su página suelta no a travez de perfil
- A las horas en reservations le falta el formato (no se sabe si es horario militar o tradicional)
- Exponen estado rejected, aunque es imposible llegar a este, porque mencionan que a futuro va a estar, Idem para estado pending. Hay funcionalidad a medias en la pagina que no esta implementada.
- Carrusel no esta paginado, Ni limitado
 - Reemplazar idealmente carrusel.
- El review estaría bueno que tenga un feedback (positivo negativo o estrellas o tags) para después poder usar.
- Para ver latitud y longitud, te redirigen a una pagina de terceros llena de publicidad.
- Seria bueno que en los espacios reservados te muestre el auto con el que reservaste

Código

- Hay bastante lógica de negocio en controllers, por ejemplo en `ParkingSpaceController`. **Esto es un error conceptual grave.**
- Hay muchas validaciones tales como en `ReviewController` (Uso pobre de bean validations) que deberían realizarse con anotaciones. Repiten esto en varios métodos. **Esto es un error**

grave.

- Los JSP imprimen valores a salida sin escapar, siendo susceptible a XSS. **Esto es un error grave.**
- En algunas instancias no escapan los caracteres especiales de LIKE.
- Tienen tests que no son verdaderamente unitarios, en capa de servicios y también en capa de persistencia. Un ejemplo se ve en `FavoriteSpaceJdbcDaoTest` al utilizar el `create()` dentro `deltestDeleteFavorite` . Se repite a lo largo de los tests. **Esto es un error conceptual grave.**
- Remember-me key hardcodeada y débil; debe ser externa y robusta.
- A lo largo del código, en los controllers, hay chequeos manuales de control de acceso. Esto viola la separación de responsabilidades y **es un error grave.**
 - Redirección manual a login, en vez de usar spring-security (Ej, bloquear únicamente el POST desde webAuthConfig)
- En la capa de negocio, al crear una reserva, se acepta el `vehicleId` y solo se verifica que exista, no se valida que el vehículo pertenezca al conductor autenticado, por lo que un usuario podría reservar usando el vehículo de otro si conoce el ID.
- Se encuentra poco (pero no nulo) texto hardcodeado no internacionalizado.
- En `ProfileController` , particularmente cuando se traen los `reservationDisplays` , causan N+1 queries. Mismo error encontrado en el método `resolveHourlyPrices` . Se deberían recolectar todos los IDs primero y traer lo necesario en una sola consulta `WHERE id IN (...)` desde la capa de persistencia, y más aún no tener esta lógica en el controller.
- Mezclan Junit 5 con Junit 4 y además declaran versiones explícitas en pom hijo en vez de heredarlas. Esto rompe centralización de versiones y agrega una dependencia adicional de testing innecesaria. **Esto es un error conceptual grave.**
- Sugerencias:
 - En la capa de servicio tiren excepciones de negocio, y no reutilicen las de Java. Tratan muchos errores como `IllegalArgumentException`.
 - Tienen métodos muy largos en la capa de servicio que arruina la legibilidad y el mantenimiento, Deberían descomponerlos en metodos privados
- `FavoriteSpaceJdbcDao.findById` hace `JOIN` con `parking_spaces` y devuelve `ParkingSpace` , copiando el mapper de otro DAO

Seguimiento

No resta puntos por seguimiento.

Nota

3

Grupo 8

Demo

- Al buscar se pierde el orden de los resultados.
- La aplicación permite cargar y completar información de licencia y DNI, pero nunca me enteré... si esto es información importante, el flujo debiera llevarme a completarlo.
- No se puede cancelar una reserva ni editarla

Código

- Existen múltiples archivos de contexto solapados, como `CLAUDE.md` y `AGENTS.md`. Debiera haber uno sólo (idealmente el segundo, que es agnóstico de la herramienta).
- Definen dos properties análogas: `javax.validation-api.version` y `javax-validation-api.version`.
- Los controllers están haciendo chequeos manuales de control de acceso. Esto atenta contra la división de responsabilidades establecidas, ya que esto corresponde a Spring Security.
- Los controllers están haciendo validaciones manuales de la data ingresada por el usuario en lugar de usar forms y Bean Validation.
- Los controllers tienen lógica de aplicación, realizando múltiples invocaciones secuenciales a servicios en lugar de consumir un Façade.
- Crearon un montón de lógica en torno a la paginación que está inherentemente mal. Intenta hacer un overfetching de la base de datos (incorrecto) al intentar asumir cuantas rows tendrá el resultset (detalle de implementación de la query, e imposible de determinar a priori) para luego filtrar en memoria de manera optimista. Todo esto debiera eliminarse y simplemente escribir las queries adecuadas para paginar y traer sólo la información necesaria. **Esto es un error conceptual grave.**
- Configuran un logger para `com.zaxxer` que no es parte de la aplicación.
- Podrían ignorar con Spring Security los recursos estáticos en lugar de simplemente definirlos como `permitAll()`
- Excelente implementación de custom access checks.
- El sistema hace un `UPDATE` para actualizar el locale del usuario en cada request, sin siquiera validar si es distinto al último guardado (y ya disponible en memoria)
- Crearon un paquete `common`, con una única clase, que se usa sólo desde `services` y perfectamente podría ser una clase interna de dicho módulo.
- El módulo de `services` tiene una compile-time dependency sobre `persistence` en lugar de depender sólo de `persistence-contracts`.
- La clase `ReviewJdbcDao` modifica en forma directa la tabla `users` en lugar de deferir en el `UserJdbcDao` como owner de la misma.

Seguimiento

No resta puntos por seguimiento.

Nota

7

Grupo 9

Demo

- El modal de login / register tiene el botón de crear cuenta mal estilado
- No se puede copiar y pegar el código de verificación
- Cuando me intercepta y me pide registrarme y hacerlo, me vuelve a mandar a la home en lugar de a la acción que quise realizar
- Mensajes inconsistentes. Obras con fecha, pero sin link de compra dicen estar "inactivas"

Código

- El proyecto pesa 32 MB. Agregan una carpeta `scraping/`, con 31 MB en `scraping/images.csv`. El scrapeo que hayan hecho para popular la base de datos no debería ser parte del código del proyecto (como peor caso, un único archivo `.sql` con todos los inserts).
- No se encontró configuración de `rememberMe()` en `WebSecurityConfig.java`. Es un requerimiento de la entrega.
- Algunas validaciones de acceso se realizan manualmente en los controller (ej. `PlayPetitionController.autocomplete`) en lugar de usar seguridad declarativa. Esto viola la separación de responsabilidades. **Es un error conceptual grave.**
- Los controllers tienen lógica de negocio, cuando esta debería estar en la capa de servicios. **Esto es un error conceptual grave.**
- Los controllers tienen lógica de validación de formularios, en vez de usar Bean Validation. **Esto es un error grave.**
- Tienen lógica de sanitización de inputs (autoboxing, `trim()`, `normalize()`, etc.) en los controllers. Justo para esto están los forms, debería implementarse esa lógica en los getters.
- En `ObraController`, el mensaje de error de `ResourceNotFoundException` está hardcodeado en español: "La obra que estás intentando ver no existe o fue retirada.". Debería ser un string dentro de `messages.properties`
- `PlayPetitionController` contiene lógica de negocio compleja, como validación de dimensiones de imágenes (`readWebpImageSize`, `validateHeaderImageDimensions`) y procesamiento de flujos de creación/edición. Tiene manipulación de bits en el controller, y muchísima lógica innecesaria.

- En `RatingController.rateProduction()` y `ReviewController.submitObraFeedback()` (no limitado a estos casos), tienen ifs para diferenciar entre requests hechas con ajax y otras. Devuelven, en base a eso, información a través de headers y `ResponseEntity` en vez de `ModelAndView`. Esta no es la forma correcta de negociar contenido en HTTP.
- Los servicios llaman a múltiples DAOs de la capa de persistencia. La manera correcta es que un servicio dependa del DAO correspondiente a su entidad y de otros servicios.
- En `UserServiceImpl.create`, se instancia directamente el objeto de dominio `new User(...)` en lugar de devolver el objeto del DAO.
- Hay varios problemas de consultas N+1. Por ejemplo (pero no reducido a), `PlayPetitionServiceImpl.searchSourceObras()` realiza una consulta al DAO de producciones por cada obra encontrada en la búsqueda y `PlayPetitionServiceImpl.loadDetails()` ejecuta 3 consultas adicionales al DAO por cada petición en la lista
- Tienen código muerto sin usos
- Tienen clases de dominio (modelos) definidos en la capa de interfaces de servicios.
- El nombre del package en ninguna de las interfaces en `ar.edu.itba.paw.persistence` se corresponde con el path de los archivos (se usa `ar.edu.itba.paw.interfaces.persistence`)
- `UserDaoImpl.searchUsers` concatena comodines `%` pero no escapa caracteres especiales como `%` o `_` en la entrada del usuario.
- Se viola la regla de exclusividad de tablas por DAO. Por ejemplo, `ObraDaoImpl.deleteById` modifica `play_petitions`, `production_reviews`, `play_ratings`, `seen_obras`, `watchlist`, `production_ratings`, `shows` y `productions`
- Para verificar inclusión en SQL, arman un string con todos los objetos de una lista, en vez de pasar la lista como parámetro de JDBC a la consulta directamente.
- Se encontró texto hardcodeado en `obras/detail.jsp`: `Compartir por mail`. Todos los textos visibles deben estar en los archivos de `messages.properties`.
- Los tests de persistencia no son unitarios. Están validando una función con la implementación de otras funciones del mismo DAO. **Esto es un error conceptual grave**
- Están usando `AtomicBoolean` para verificar si se llama a un método, queriendo simular el comportamiento de `Mockito.Verify`. **Esto es un error conceptual grave**
- Ejecutan queries SQL directamente en `WebConfig`. **Esto es un error conceptual grave**
- Utilizan `WebConfig` para ejecutar migraciones manuales de datos. Estas migraciones se ejecutan cada vez que se inicia el application container, y es lógica inherentemente de persistencia.
- La lógica para "migrar" las contraseñas de texto plano a hasheadas con BCrypt, más allá de que no debería existir, es errónea. Si pongo mi contraseña como `$2a$holá` la deja como texto plano.
- Se configura la ruta para HTTP 403 directamente en `WebConfig`. ¿Por qué no lo hacen en `WebSecurityConfig`?
- El `failureHandler` en `WebSecurityConfig` tiene lógica de negocio, que debería ser abstraída.

Seguimiento

No resta puntos por seguimiento.

Nota

3

Grupo 10

Demo

- No escapan % ni _
- cuando pongo precio especial por fecha se me pisa el precio por defecto
- Estaría bueno que se pueda compartir comprobante por pdf
- Cuando no hay CBU no llega el comprobante pero la UI no indica nada de eso
- Las acciones terminales como eliminar o pausar deberían tener modal
- Falta un CTA y un mensaje en el search de users y productos del admin

Código

- `commons-fileupload` está declarado con `<version>1.5</version>` hardcodeda dentro de `webapp/pom.xml` en vez de centralizarse en el `<dependencyManagement>` del padre. Todas las versiones de terceros tienen que vivir en el POM padre para evitar drift entre módulos y para que un upgrade sea una sola edición.
- El POM padre define `javax.servlet-api` en `<dependencyManagement>` sin `<scope>provided</scope>`. La declaración del scope `provided` tiene que estar una sola vez en el padre para garantizar que ningún módulo termine empaquetando el jar del servlet API al WAR final (el contenedor ya lo aporta).
- `DashboardController.computeReviewableRentIds` decide en el controller qué rents son reseñables. La regla "rent reseñable = finalizada y no reviewed todavía" es de dominio y tiene que vivir en `ReviewService`. Esto es lógica de negocio en el controller y **es un error conceptual grave**.
- `CatalogController.parseCategoryFilters` interpreta los query params aplicando reglas de dominio dentro del controller: `Category.usesSize()` para decidir si la categoría lleva size, `Size.isValidForCategory(category)` para descartar combinaciones inválidas. La traducción de params a un `CatalogCriteria` con esas reglas es lógica de búsqueda y pertenece al service
- `ExceptionHandler` mapea `/error/403`, `/error/404` y `/error/500` con `@RequestMapping("/error/403")` sin declarar el `method`
- `ImageController.detectContentType` inspecciona magic bytes del archivo (PNG: `89 50 4E 47`, JPEG, GIF, WebP) para deducir el `Content-Type` que devuelve al cliente. Eso es procesamiento binario que pertenece a un servicio o componente especializado, no al controller.

- `CatalogController.buildFilterQueryString` serializa el query string de filtros con `StringBuilder` y `appendMulti / appendParam` y lo expone como `${filterQueryString}` para que la vista lo concatene en la URL de paginación. Doble problema: el browser ya serializa eso solo si se envuelve search-bar+filter-panel+sort en un único `<form action="/catalog" method="get">`, y los valores se concatenan SIN `URLCoder.encode`, así que un search/category con `&`, `=` o espacios rompe los links de paginación. Reemplazar por un `<form method="get">` o usar `<c:url><c:param/></c:url>` que ya hace URL-encoding.
- La validación de **ownership** está implementada manualmente dentro de los services en vez de centralizarse en la capa de seguridad. Esto viola la separación de responsabilidades. Si bien la programación defensiva dentro del servicio es deseable, el acceso debe restringirse antes a la entrada.
- Hay métodos que iteran un ciclo llamando multiples veces a la base de datos, esto es un potencial problema de performance.
- En `ReviewJdbcDao` el `RowMapper<Review>` se define como lambda anónima inline en `findByProductId` y se duplica casi idéntico en `save`. Conviene extraer un `private static final RowMapper<Review> REVIEW_ROW_MAPPER = (rs, n) -> { ... };` y reusarlo desde ambos métodos, como ya lo hacen los demás DAOs del proyecto
- `ReviewJdbcDao.findByProductId` hace `JOIN` a la tabla `users` y reconstruye una entidad `User` (con builder) inline dentro de su `RowMapper`, duplicando la lógica de mapeo que ya vive en `UserJdbcDao`. Si mañana cambia un campo de `User` el mapper de Review queda desincronizado en silencio y no hay test que lo agarre. Los RowMappers se pueden compartir y reutilizar para evitar esto.
- No escapan `%` ni `_` en el search
- Los tests unitarios en general tienen **graves problemas**:
 - Cuatro tests en `ProductServiceImplTest` usan `AtomicReference` dentro de `doAnswer` para capturar los argumentos pasados al mock y luego hacer asserts sobre ellos: `testCreateProductWhenValidReturnsProduct` captura las bytes pasadas. Es reimplementar `Mockito.verify`. Esto verifica la implementación y no el comportamiento.
 - Hay tests del service que solo validan `Assertions.assertDoesNotThrow(...)` sin validar el comportamiento prometido por el nombre del test.
 - `CurrencyTest.testFromSlugWhenAllValuesRoundtrip` ejecuta `Currency.fromSlug` trs veces (una por cada valor del enum) en un único `@Test`. Si falla para alguno, JUnit reporta el método entero sin indicar qué valor falló, y un fallo aborta el loop tapando otros posibles fallos. Convertir a `@ParameterizedTest` con `@EnumSource(Currency.class)`.
- Los aria-labels están hardcoded en ingles, deberían tambien estar internacionalizados

Seguimiento

No resta puntos por seguimiento.

Nota

6

Grupo 11



Este grupo deployó durante la demo de los otros grupos. Como penalización, se les hizo mandar un mail con el commit asociado al último deploy, y se decidió tomar el horario de ese correo como horario de entrega:

lun, 4 may, 22:05

Demo

- Se les pasaron algunas palabras que no están internacionalizadas
- No puedo ver los detalles de los kayaks si no estoy logueada
- No validan el email al crear una cuenta
- El formulario de creación de bote se resetea cada vez que se agrega una foto. La preview de la foto no anda.
- El email de que mi publicación esta live me lleva a `"/profile"` y no a `"/my-boats"`. También esta mal enviada la URL
- El buscador de Reservas o de Mis barcos no está conectado. No hace nada. El filtro de estado tampoco.
- La página del marketplace (página 1) tarda demasiado en cargar. Es posible que hagan filtros en Java: Cargar los barcos tarda como 10 segundos cuando hay 12 barcos
- El "save draft" de la creación, guarda la data en LocalStorage, no hace nada contra el backend. Esto puede sobrecargarse (LocalStorage tiene un máximo de 5 MB) y no es una funcionalidad bien implementada.
- Le falta el nombre de la página en la que estás parado.
- El pdf viewer del comprobante de pago está roto.
- Formulario de creación de bote no valida en backend un número muy grande (precio por hora), tiran un 500 (lo catchean).
- Feature "extra" (entrega tardía, commit que pushearon una hora mas tarde). Son las reviews. Parece funcionar ok.

Código

- Se declaran explícitamente las versiones de los plugins en los POMs hijos (`webapp/pom.xml`, `services/pom.xml`, `persistence/pom.xml`, etc.) en lugar de centralizarlas en `pluginManagement` del POM padre.
- Archivos de configuración de IDE están siendo versionados (ej. `.vscode/settings.json`), lo cual no corresponde al proyecto.
- Remember-me key hardcodeda y débil; debe ser externa y robusta.

- Existen validaciones manuales de pertenencia / estado en las clases de soporte de controladores como por ejemplo en `PublishMvcSupport` y `GalleryMvcSupport`). **Esto es un error grave**, dado que las reglas de autorización deben resolverse de forma declarativa mediante Spring Security.
- Mucha de la lógica de negocio se ha encapsulado en clases "MvcSupport". Las funciones de soporte tienen lógica de negocio, que debería estar encapsulada en los servicios y no con los controllers (ej. `PublicationAvailabilityMvcSupport.blockSlot()`). **Esto es un error grave**.
- Hacen chequeos de login y en base a eso redirigen a login. Eso se debe delegar en Spring Security.
- Ausencia y uso inconsistente de anotaciones `@Transactional` . Varios servicios carecen de estas anotaciones en métodos que leen y escriben en la base de datos.
- Paginan en memoria, en vez de utilizar la base de datos con cláusulas `LIMIT` y `OFFSET` . **Esto es un error grave**.
- Cuando hacen búsquedas en el `ItemServiceImpl` con fechas, los filtros los hacen en memoria. **Esto es un error grave** y es el causante de que la página cargue lento en algunas instancias.
- Los servicios llaman a múltiples DAOs de la capa de persistencia. La manera correcta es que un servicio dependa del DAO correspondiente a su entidad y de otros servicios. Por ejemplo, `ReviewServiceImpl` depende de `ItemDao` , `ItemBookingDao` , `ReviewDao` y `UserDao` ; debería depender de `ItemService` , ..., `ReviewDao` y `UserService` .
- Se producen varios "joins en Java" y problemas de consultas N+1. **Esto es un error conceptual grave**. Por ejemplo (pero no reducido a), en `ReviewServiceImpl.getItemRatingSummaries` se itera la lista llamando a `getItemRatingSummary(itemId)` de a uno, generando una consulta por cada elemento.
- Se viola la regla de exclusividad de tablas por DAO. `ItemMediaJdbcDao` modifica (hace un UPDATE) la tabla `item_publication_version` a través de un helper (`ItemPublicationCoverJdbcSupport`), cuando esta tabla ya es mantenida y actualizada por `ItemJdbcDao` .
- En las pruebas de persistencia (ej. `UserJdbcDaoTest`), no se está validando de manera fehaciente el estado final de las tablas tras las operaciones.

Seguimiento

No resta puntos por seguimiento.

Nota

4

Grupo 12

Demo

- La página explota desde el cliente cuando se intenta enviar un form con `<script>alert(document.cookie)</script>` (lo particular es `document.cookie`). Se queda colgada. Están evaluando el JS antes de mandar el form?
- Inyectan HTML después de una request a la "API" para hacer el infinite scroll. En general en toda la página se intenta no recargar, como si fuera una SPA. El frontend hace el request e inyecta el HTML devuelto en el DOM. Esto no necesariamente está mal si está correctamente implementado.
- Se puede hacer doble submit para ser moderator.
- El redirect después de submittear moderador tira 403
- El CTA del email cuando me aprueban para ser moderador tira 403
 - El usuario debe hacer logout para que se actualice el rol
- La navbar a veces esta inconsistente - desaparece la opción de Admin
- Roto el confirmar auto por las imágenes - "no probamos el flujo en producción"
- UX:
 - Espacios vacíos en las landing
 - Paginación inconsistente
 - Falta el año del auto en "Cars that fit you"
 - En "Cars that fit you" las cards están inconsistentes, y mal espaciadas
 - Los tags no son clickeables
 - Las imágenes en los emails no se están mostrando bien
- En el "Cars that fit you", que se llega con la encuesta de la landing, tienen mal los URLs de las cards. Les faltó el `c:url`.
- Los paths de la página usan query params para navegar, en vez de paths.
<http://pawserver.it.itba.edu.ar/paw-2026a-12/reviews?carId=526>
- Buena página cuando no hay autos que hagan match

Código

- El módulo `services` declara `<maven.compiler.release>17</maven.compiler.release>`, sobrescribiendo la versión Java 21 definida en el POM padre.
- La dependencia `spring-jdbc` está declarada en `webapp` con scope `compile`. La capa `webapp` no debería tener dependencia directa de JDBC; ese acceso pertenece al módulo `persistence`.
- El método privado `isAdmin()` inspecciona roles directamente en el controller `CarReviewController`. La verificación de roles debe delegarse a Spring Security, no realizarse en el controller. **Esto es un error conceptual grave.**
- En el servicio `WeeklyDigestServiceImpl` se invoca `carRequestService.getCarRequestsByStatus(CarRequestService.STATUS_PENDING).size()` para

obtener un conteo. El servicio ya expone `countCarRequestsByStatus()`, que evita cargar la lista completa en memoria solo para contar.

- Se realizan varios filtros en memoria cuando deberían delegarse a SQL, como por ejemplo (pero no reducido a) en `WeeklyDigestServiceImpl` cuando se recuperan todas las reseñas de los autos favoritos y luego se filtra en memoria por fecha. **Esto es un error grave.**
- Se presenta el patrón N+1 en varias instancias, como por ejemplo (pero no reducido a) en `CarServiceImpl`, el método `collectRetainedImagePayloads()` itera sobre `retainedImageIds` y llama a `findByCarIdAndImageId()` por cada elemento. **Esto es un error conceptual grave.**
- En `CarServiceImplTest` se utiliza `verify()` para verificar la interacción con el mock. El uso de `verify()` está explícitamente prohibido en los tests de servicios ya que valida la implementación y no el comportamiento. **Esto es un error conceptual grave.**
- Los archivos de esquemas contienen sentencias de migraciones de datos mezclados con DDL estructural.

Seguimiento

No resta puntos por seguimiento.

Nota

5

Grupo 13

Demo

- Hay modales que tienen muchísimos inputs, mejor evitar tener muchos inputs en los modales ya que no se dan bien a refrescos o procesos multi step.
 - Ese input me tira un error de max characters cuando en realidad el error era de regex (dirección).
- Tendría que haber manera de guardar direcciones para envíos múltiples.
- Bien el engine de recomendaciones, pero también estaría bueno agregar un factor de compra
- No puedo ver follows y following.
- Bien de features.
- DatePicker pierde consistencia con la página.
- La internacionalización está mal hecha, (a mano vía un queryParams). Tienen que utilizar Accept-Language o en su defecto un override manual desde la UX (No respetan negociación de contenido http)
 - Al menos el mail sí que llega internacionalizado.
- Bien los controles de acceso

- Muy bien de features.
- Bien los flujos principales.

Código

- Inconsistencia entre POM padre e hijos en la versión target del compilador Java provoca builds con bytecode mezclado entre módulos (Ej: `pom.xml:18-19` declara `maven.compiler.target=21` mientras `models/pom.xml:17` lo redeclara como `maven.compiler.release=17`)
- Versión de plugin Maven redeclarada en hijo con valor distinto al del padre genera inconsistencia de build (Ej: `pom.xml:33` con `<maven-war-plugin.version>2.6</maven-war-plugin.version>` redeclarado como `3.4.0` en `webapp/pom.xml:137`)
- Versión literal duplicada en `dependencyManagement` con una property existente del mismo valor, propenso a divergir (Ej: `pom.xml:48-50` importa `junit-bom` con `5.11.0` mientras existe `${junit.version}=5.11.0`)
- Archivos vacíos sin justificación versionados en múltiples módulos ensucian el repo (Ej: `models/.mvn/jvm.config` y `models/.mvn/maven.config` vacíos)
- Capas superiores dependen directamente del módulo de implementación saltándose la abstracción de contratos (Ej: `webapp/pom.xml:67-70` declara dependencia a `persistence` en vez de solo `persistence-contracts`)
- Secretos hardcoded en código fuente en lugar de externalizarse a properties o variable de entorno los expone en VCS y los hace forjables (Ej: remember-me key en `webapp/.../WebAuthConfig.java:58` con `.key("mysupersecretket...")`)
- Lógica de autorización duplicada entre capas en lugar de centralizar en un solo componente genera reglas de seguridad divergentes (Ej: `webapp/.../auth/ArtworkSecurity.java:14-71` duplica `isArtworkOwner/isPurchaseBuyer/isPurchaseSeller` ya presentes en `services/.../SecurityServiceImpl.java`)
- Inyección del user autenticado redeclarada idénticamente en múltiples controllers en vez de centralizarse en un `@ControllerAdvice` global (Ej: método `@ModelAttribute("loggedUser")` con `@AuthenticationPrincipal` en `webapp/.../ProfileController.java:32-34`)
- Catches inconsistentes dentro del mismo archivo, uno genérico que traga la excepción y otro con tipos específicos, confunden el contrato de errores (Ej: `webapp/.../GlobalExceptionHandler.java:96-100` con `catch (Exception)` vs `userHasPurchases:56-65` con `DataAccessException` + `RuntimeException`)
- Mensajes de error hardcoded en inglés dentro del controller en lugar de bundle de i18n rompe traducciones (Ej: `webapp/.../RegisterController.java:120-121` con `rejectValue("That username is already in use...")` y `:124` con `"That email is already in use..."`)
- Múltiples métodos de escritura en services no están anotados con `@Transactional` , pudiendo dejar la BD inconsistente ante fallos parciales (Ej: `services/.../PurchaseServiceImpl.java:65` `processPurchase`)

- Métodos solo-lectura sin `@Transactional(readOnly=true)` desaprovechan optimizaciones del TX manager y no documentan intención (Ej: `services/.../ExploreViewServiceImpl.java` `getExploreData` sin anotación)
- Borrado en cascada con múltiples DELETES ejecutados desde el DAO sin transacción que envuelva la operación pone en riesgo el dejar datos huérfanos (Ej: `services/.../ArtworkServiceImpl.java:189-206` `deleteArtwork` dispara 5 deletes separados vía `ArtworkJdbcDao.deleteById`). **Esto es un error grave.**
- DAOs que operan sobre tablas ajenas a su entidad rompen la exclusividad de tabla por DAO y generan múltiples puntos de acceso a la misma tabla (Ej: `persistence/.../ArtworkJdbcDao.java:552` ejecuta `DELETE FROM purchases WHERE artworkId = ?` , mezclándose con `PurchaseJdbcDao`). **Esto es un error conceptual grave.**
- Paginación en memoria sobre carga completa del catálogo con `Integer.MAX_VALUE` + `subList` no escala — las queries subyacentes (`getCandidateArtworkIds` , `getEligibleArtworkIdsForSuggestions`) crecen con el tamaño del marketplace, no con el usuario (Ej: `services/.../RecommendationServiceImpl.java:155` `getSuggestedArtistsPage` y `:161` `getSuggestedArtworksPage`). **Esto es un error grave.**
- Combinación de chequeos resueltos con múltiples llamadas individuales al DAO candidatas a join/batch (Ej: `services/.../ProfileViewServiceImpl.java:50-53`)
- Upsert implementado como UPDATE-then-INSERT a nivel aplicación tiene un posible race condition TOCTOU (Time-of-check to time-of-use), la solución correcta es `INSERT ... ON CONFLICT DO UPDATE` de Postgres o `SELECT ... FOR UPDATE` previo (Ej: `persistence/.../SellerJdbcDao.java:46-58` `saveOrUpdate`)
- Lectura de columnas nullable con `rs.getLong()` borra silenciosamente los NULLs (los convierte en `0L`), contradiciendo el schema y la lógica del service que sí chequea null, bug latente en el motor de recomendaciones (Ej: `techniqueId / mainColorId`)
- Modelos sin `equals/hashCode` por ID hacen que dos instancias con mismo `id` no sean `equals` , rompiendo `Set / Map / contains` en lógica de negocio (Ej: `models/.../Artwork.java` , `Purchase.java` , `User.java` no implementan ninguno)
- Buena cobertura de tests
- Tests con `AtomicReference` para capturar/verificar invocaciones a pesar de no llamarse `verify` testean implementación. **Esto es un error conceptual.**

Seguimiento

No resta puntos por seguimiento.

Nota

5

Grupo 14

Demo

- Bien que verificar me deja logueado
- Buena la integración con openstreetmap
- Ojo con la visualización de la opción de "Después de cierta cantidad de fechas" en la creación de un evento, se corta el texto
- Falta un CTA a la página cuando me llega por mail una solicitud de ingreso o invitación
- El botón de aplicar filtro de fechas, el botón de precio o el campo de texto en "Mis partidos" redirige a una página con la url mal formada (falta /paw-2026a-14)
- Número de página negativo en /events tira 500
- % y _ no están escapados en las búsquedas por texto
- Sacar el mapa cuando no se completó ese campo
- La localización va por query param
- En la página de 404 me aparece la barra de usuario anónimo cuando ya estoy logueado
- Faltaría una vista o redirección desde el detalle de un reporte al detalle de una vista
- El motivo de baneo no está internacionalizado
- Hay un problema con la zona horaria, e.g. al editar la locación de un evento, no se mantuvo las 9pm sino que se pasó a las 00 (+3)
- Bien el chequeo de pasar de participación por confirmación a evento público

Código

- Commitearon `webapp.war` y directorio `target`. Eventualmente los sacaron y los agregaron al `.gitignore`.
- Agregan `spring-webmvc` al bloque `dependencies` del pom padre (no `dependencyManagement`), haciendo que todos los pom hijos lo hereden. **Esto es un error conceptual.**
- La dependencia `jackson-databind` no corresponde al módulo models: no la usan y este módulo no debiera manejar serialización.
- Definen algunas versiones de las dependencias en pom hijos en vez de centralizarlos en el pom padre.
- No queda clara la utilidad de `ShellViewModelFactory.java`, la renderización de acuerdo a los roles del usuario se puede lograr en jsp con la taglib de spring security sin necesidad de generar componentes en java.
- Su capa webapp accede de forma directa al `UserBanDao` (en `SecurityConfig` y `BannedAccountAuthorizationFilter`) sin pasar por un servicio rompiendo el modelo de capas jerárquicas. **Esto es un error conceptual grave.**
- Tienen mucha localización en los controllers y en los servicios (a través de `messageSource.getMessage(...)`) en vez de relegarlo a `spring:message`.

- Mezclan `@PreAuthorize` y configuración de seguridad en `SecurityConfig`, sería mejor elegir uno y seguir una convención. Idealmente, el uso centralizado de `SecurityConfig` permite tener todas las reglas de acceso en un sólo lugar.
- No tienen configuración para el `rememberme` key.
- Usando `webSecurityCustomizer` y `web.ignoring()` se ahorrarían duplicar el manejo de los archivos estáticos.
- Tienen varias validaciones de formularios en los controladores (por ejemplo, la comparación de contraseñas en el formulario de registro), cuando lo correcto sería que lo manejen con validators desde los formularios. El manejo de sus errores tampoco es adecuado: esperan que se lancen excepciones con un string de código hardcodeado que comparan para saber qué error mostrar. **Estos son errores conceptuales graves.**
- Las funciones de “normalización” se encuentran duplicadas y no son necesarias, bastaría con usar el tipo apropiado en los parámetros de los controllers, por ejemplo para recibir números, enums; y dejar que Spring resuelva.
- Tienen lógica de seguridad/permisos implementada en los controllers, por ejemplo el método `isMatchVisibleToUser`. Esto corresponde a Spring security y rompe la división de responsabilidades.
- Por más de que sean actualmente números chicos, la búsqueda de partidos, filtrado y deduplicación en el `MatchDashboardController.java` no es apropiada. Tampoco es correcta la repaginación. Debería ser una query a la db con filtros, offset y limit.
- En `ModerationServiceImpl`, para buscar si ya existe un reporte similar, se traen una lista de tamaño `Integer.MAX_VALUE` y hacen un `.anyMatch`. Similar en otras queries, filtran en memoria en vez de delegarlo al DAO. **Esto es un error grave**
- Tienen instancias de queries N+1: buscan ids que cumplan ciertas condiciones y luego iteran por cada una para operar o traerse la entidad, e.g.
`MatchParticipationServiceImpl.findPendingRequestMatches` ,
`ModerationServiceImpl.cancelFutureContentForUser` .
- Tienen DAOs que son accedidos por varios servicios, por ejemplo `ModerationServiceImpl` usa `MatchDao` , `PlayerReviewDao` , `UserDao` .
- Tienen casos donde el servicio define adónde redirige la acción, esa decisión es del controller.
- Configuran `ThreadPoolTaskExecutor` pero sin `setWaitForTasksToCompleteOnShutdown` o `setRejectedExecutionHandler`.
- Encapsulan/Abstraen el contenido de los mails en clases `...TemplateData` pero terminan siendo las clases externas que llaman al servicio de mailing las que deben transformarlas en un `MailContent` para poder llamar a la única función de mailing ofrecida. De esta forma, el `MailDispatchService` termina no siendo dueño de la gestión de los mails, sino que se delega el saber qué template es necesario para cada mail y cualquiera podría enviar un mail cuyo contenido no esté estandarizado.

- El uso de `replaceFirst("SELECT \\ ", "SELECT COUNT( ")` acopla su implementación de count a que su otra query se mantenga igual.
- En sus archivos de migraciones de db falta V16, crean tablas que no son manejadas por ningún DAO, y sus scripts de test no se corresponden con los de prod, lo que dificulta entenderlos.
- Sus tests de views sólo validan que los archivos contengan cierto texto explícito. Están fuertemente acoplados a la implementación y no informan acerca del comportamiento. Lo mismo sucede con `ThymeleafMailTemplateRendererTest`.
- Su `UiRouteTest` tiene un setup de 1400 líneas para una implementación custom de los servicios cuando podría ser un mock.
- Tienen tests que usan AtomicReference, AtomicBoolean o similares para verificar llamados a métodos mockeados. Esto testea (erróneamente) implementación. **Esto es un error conceptual grave.**
- Usan reflection para acceder al ID de `auth.getPrincipal()`. Esto los ata a un nombre de función en específico y es innecesario.
- Tienen lógica de negocio en los controllers. Por ejemplo, en `HostControoller.publishEvent` es el controlador el que orquesta la creación de la imagen y del match, y en `EventController` detallan qué permisos tiene un usuario sobre un evento. **Esto es un error conceptual grave.**
- Crean DTOs en el sentido servicio → jsp, variando entre clases y records, que luego desglosan para las vistas o usan para popular abstracciones de componentes. Sus modelos son suficientes, están perdiendo funcionalidad del Expression Language y esparciendo las vistas en .jsps, clases y controllers.

Seguimiento

No resta puntos por seguimiento.

Nota

3